Online Order Processing with the Facade Pattern

Assignment 4 | Software Design Patterns | Java 17

Course ShP-2216 | Option B | 17 October 2026
Prepared for repository owner **bossmerej-ship-it**

1 Introduction

An online checkout is one user action with several dependent steps. Stock must be available, payment must succeed, and a courier must accept delivery before the order can be confirmed. If each client implements this sequence, it must also implement the matching rollback rules. The same coordination would be repeated in a console, website, or mobile app.

This project implements the assignment's suggested Option B. **OrderFacade** exposes **placeOrder(cart, customer)**, **cancelOrder(orderId)**, and **listProducts()**. The client supplies domain data and receives a receipt; it does not build or call any of the four subsystem services. The facade provides a simpler entry point to a complex workflow, following the Facade structure described by Refactoring.Guru [1].

The subsystem performs observable work

InventoryService calculates totals from a catalog, validates all cart items, updates stock, and records reservations. **PaymentProcessor** records decimal charges and refunds. **ShippingService** validates courier coverage and records tracking numbers. **NotificationService** formats messages and stores them in a local outbox.

The services collaborate through the facade using the same order UUID, cart, calculated amount, and customer. They are stateful simulations rather than external integrations. There is no real card charge, email delivery, or courier request.

A simpler client

```
OrderFacade shop = new OrderFacade();
OrderReceipt receipt = shop.placeOrder(cart, customer);
OrderReceipt cancelled = shop.cancelOrder(receipt.orderId());
```

Without the facade, a client must price the cart, reserve inventory, charge payment, schedule shipping, notify the customer, and compensate earlier steps on failure. The three client calls above replace that orchestration, while the services remain available for advanced direct use.

2 UML class diagram

The application client depends on the facade. The facade has one-way associations to four subsystem services. No service contains a reference to OrderFacade.

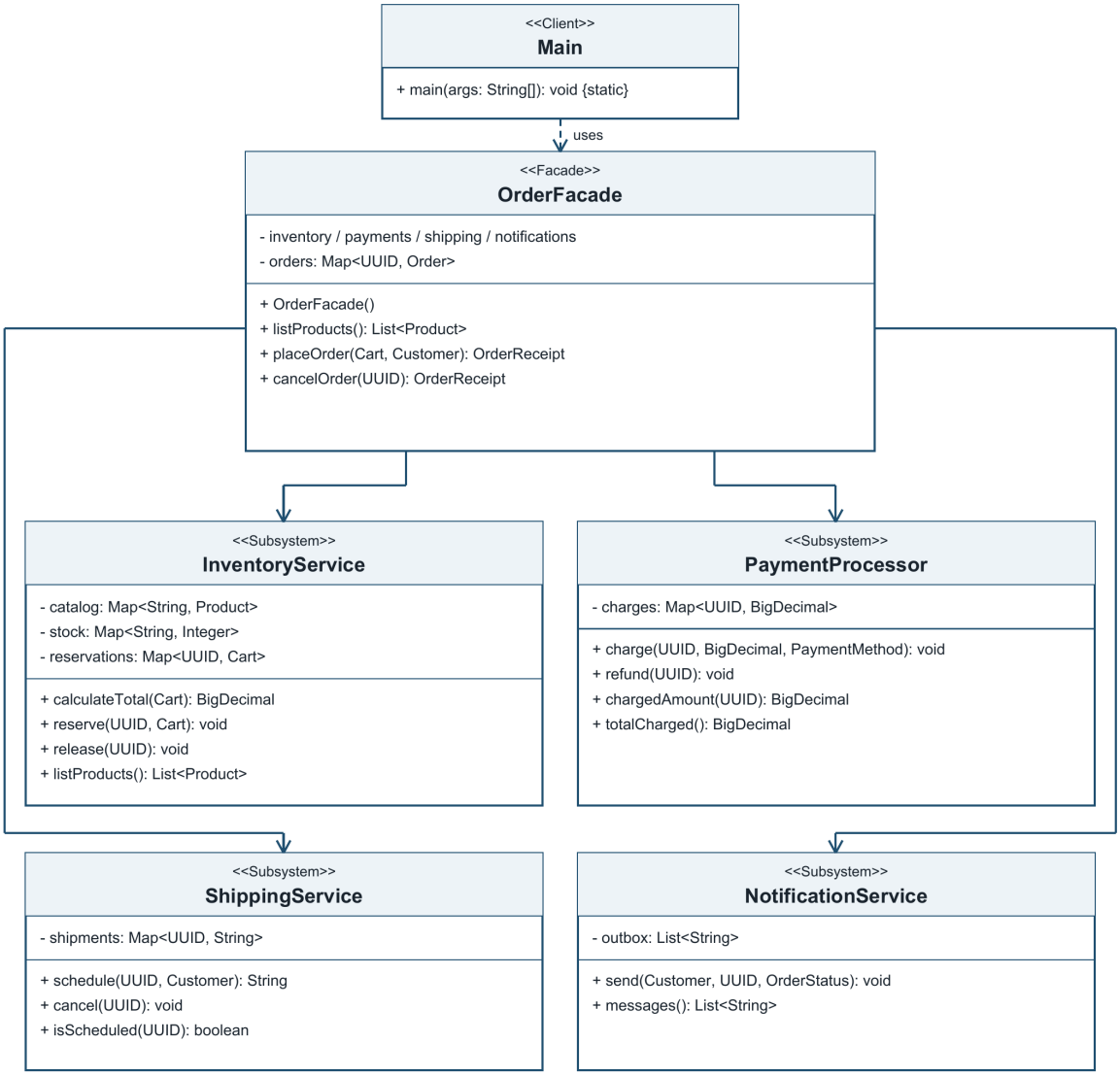


Figure 1. Core classes, private state, and principal public operations. + means public and - means private. Selected accessors are omitted for readability.

Data contracts and independence

Cart, Customer, Product, and OrderReceipt are Java records; PaymentMethod and OrderStatus are enums. These domain values cross the facade boundary. OrderException preserves workflow failure diagnostics. The complete editable diagram, including domain dependencies and helper methods, is provided in docs/UML.puml.

The public no-argument constructor creates the default subsystem. A package-private constructor allows tests to inject controlled failures. Each service has public methods and can be instantiated without a facade; the directSubsystemUse test verifies this.

3 Clean Code principles

The following six principles are illustrated with short excerpts from the implementation. The annotations explain why each choice helps this workflow.

1 Meaningful names and a small API

The public methods describe business actions. The caller does not need to learn reservation IDs, gateway commands, or shipment construction.

```
OrderReceipt receipt = shop.placeOrder(cart, customer);
shop.cancelOrder(receipt.orderId());
```

Annotated excerpt from Main: placeOrder and cancelOrder communicate intent; OrderReceipt names the returned business result. The only additional operation is the catalog query listProducts.

2 Single Responsibility and focused services

The facade coordinates. Inventory owns pricing and reservation rules, payments own charges, shipping owns coverage and tracking, and notifications own messages.

```
total = inventory.calculateTotal(cart);
inventory.reserve(orderId, cart);
payments.charge(orderId, total, customer.paymentMethod());
trackingNumber = shipping.schedule(orderId, customer);
```

Annotated excerpt from OrderFacade: business steps are delegated in order. The facade does not subtract quantities, calculate line prices, or build a tracking number.

3 Encapsulation and immutable boundaries

The facade returns domain values rather than internal service objects. Input collections are copied so a caller cannot silently change an existing reservation.

```
// Cart compact constructor, after validating every item:
items = Map.copyOf(items);
// OrderFacade stores its collaborators privately:
private final InventoryService inventory;
```

Annotated excerpts from Cart and OrderFacade: the defensive copy protects the cart from later changes to the caller's map. Private final service fields prevent normal clients from reaching the subsystem through the facade.

Clean Code principles continued

4 Fail fast before changing shared state

Inventory checks every item before subtracting any stock. A cart containing an available book and an unavailable USB drive must not reserve just the book.

```
for (var item : cart.items().entrySet()) {
    requireProduct(item.getKey());
    if (stock.get(item.getKey()) < item.getValue()) {
        throw new IllegalStateException(
                "Insufficient stock for " + item.getKey());
    }
}
```

Annotated excerpt from InventoryService.reserve: the entire validation loop finishes before the later subtraction pass. Cart and Customer also reject invalid values at construction.

5 Explicit error handling with preserved diagnostics

A later failure triggers reverse compensation. Cleanup must continue even if one compensating action throws, and the initial cause must remain inspectable.

```
try {
    action.run();
} catch (RuntimeException rollbackFailure) {
    failure.addSuppressed(rollbackFailure);
}
```

Annotated excerpt from OrderFacade.compensate: every action is attempted in shipping, payment, inventory order. The wrapper exception keeps the initial cause; suppressed exceptions retain cleanup errors. Notification failure instead sets a receipt flag after business success.

6 Idempotent compensation and safe retries

A repeated cancellation must not increase stock twice or refund twice. Services remove each recorded effect once; a second cleanup finds nothing to reverse.

```
Cart cart = reservations.remove(orderId);
if (cart != null) {
    cart.items().forEach((sku, quantity) ->
        stock.put(sku, stock.get(sku) + quantity));
}
```

Annotated excerpt from InventoryService.release: removing the reservation makes release idempotent. Payment refunds and shipment cancellation also remove records. The facade returns the stored receipt for an already cancelled order.

4 Validation and demonstration

The project was compiled and executed with JDK 17.0.12 using **--release 17 -Xlint:all -Werror**. All **19 behavioral tests** pass. Tests throw AssertionError on failure without requiring the JVM -ea option. No third-party Java library is required.

The automated checks cover successful multi-item checkout, declined payment, delivery failure, atomic reservation, unknown products, input validation, cancellation, repeated cancellation, separate orders, immutable snapshots, exact decimals, notification outage, cancellation retry, preserved cleanup errors, direct subsystem use, duplicate protection, and the simplified public API.

Live sequence: run **./run.ps1 demo** on Windows or **bash run.sh demo** on Linux/macOS. One BOOK at 12,500 KZT and two USB drives at 3,500 KZT total **19,500.00 KZT**. Stock moves from BOOK=5, USB=10 to BOOK=4, USB=8. Cancellation restores 5 and 10; repeating it leaves these values unchanged. The demo then exercises declined payment, unsupported delivery, and insufficient stock.

Use **./run.ps1 test** or **bash run.sh test** for the checks, and select **interactive** instead of **demo** to use the console menu. A declined card uses DECLINED_CARD. Country KZ succeeds; country US causes a shipping failure after payment, exercising both refund and stock release.

5 Conclusion and implementation limits

The facade hides subsystem creation, operation ordering, shared order identifiers, compensation, and notification policy. The client retains direct access to business data and receipts. Advanced clients may independently use any subsystem service; the services do not depend on the facade. This preserves the defining Facade property [1].

This is a single-threaded, in-memory simulation. Active orders retain reservations; fulfilment and completed delivery are outside scope. State resets when the process exits. Real deployments need persistence, concurrency control, integration adapters, and durable recovery for failed compensation. The simulation preserves those failure diagnostics but provides no background recovery or notification retry.

6 Repository and references

GitHub repository: https://github.com/bossmerej-ship-it/SDP

[1] Refactoring.Guru. Facade. refactoring.guru/design-patterns/facade. Accessed 17 October 2026.

[2] Oracle. Java SE 17 BigDecimal API and Record Classes. Decimal arithmetic and record semantics follow the Java 17 documentation: BigDecimal; Record Classes. Accessed 17 October 2026.

[3] Assignment_4_Facade_Pattern_Instructions.docx, supplied assignment brief. The supplied student repository was reviewed as a structural example; this project uses Option B and independently written code.